```
def spawn_population(length=26,size=100):
5
    pop = []
    for i in range(size):
        string = ''.join(random.choices(alphabet,k=length))
        individual = Individual(string)
        pop.append(individual)
       return pop
```

- ***1*** The list of characters we sample from to produce random strings
- ***2*** The string we're trying to evolve from a random population
- ***3*** Sets up a simple class to store information about each member of the population
- ***4*** Computes a similarity metric between two strings, giving us a fitness score
- ***5*** Produces an initial random population of strings

The preceding code creates an initial population of individuals which are class objects composed of a string field and a fitness score field. Then it creates the random strings by sampling from a list of alphabetic characters. Once we have a population, we need to evaluate the fitness of each individual. For strings, we can compute a similarity metric using a built-in Python module called `SequenceMatcher`.

In listing 6.2, we define two functions, `recombine` and `mutate`. As their names suggest, the former will take two strings and recombine them to create two new strings, and the latter will randomly flip characters in a string to mutate them.

Listing 6.2. Evolving strings: recombine and mutate

```
def recombine(p1_, p2_):                    1
    p1 = p1_.string
    p2 = p2_.string
    child1 = []
    child2 = []
    cross_pt = random.randint(0,len(p1))
    child1.extend(p1[0:cross_pt])
    child1.extend(p2[cross_pt:])
    child2.extend(p2[0:cross_pt])
    child2.extend(p1[cross_pt:])
    c1 = Individual(''.join(child1))
    c2 = Individual(''.join(child2))
    return c1, c2
```